memo

主な処理

ファイルの読み込み

- 以下の形式のファイルを染色体A, Bとして解釈し、読み込む。

```
6
1 3 -2 -5 ^ -6 4
1 2 ^ 3 4 5 ^ 6
```

備考

- 1行目：geneの数 n.
- 2行目：chromosome A. 符号は'-'のみ許容。' 'により区切る。'^'は、その染色体の切れ目を表す。便宜上「切れ目」と表現しているが、厳密にはその前後は「別の染色体」である。In other words, '^'の数 + 1個の染色体が含まれると言える。なお、そこまで厳密にエラーハンドリングしていないため、行頭と行末に'^'が記載されている場合はおそらくクラッシュする。
- 3行目：chromosome B. 書式は2行目と同様。切れ目はAのそれらと全く関係なく配置されている可能性があるが、1からnまでは順に並んでおり、符号はすべて+。これはgeneのラベリングの自由度により、Bのgeneの番号は任意に決められることに因る。また、複数のchr. から複数のchr. への距離を考えるため、Bも切れ目が入っていることがある。

実装

/include/genome.h

```
typedef struct Extremity_ctx
{
    Extremity **exts;
    Extremity *ext_l;
    Extremity *ext_r;
    Extremity **goals;
    Extremity *goal_l;
    Extremity *goal_r;
    int n;
    int *perm;      [1, 3, -2, -5, -6, 4]
    int *sigma;     // 使っていません
    int n_sent;     10
    int nb_sent;    12
    int *perm_sent; [0, 1, 3, 2, 5, 7, 0, 6, 4, 7]
    int *sigma_sent;[1, 1, 1,-1,-1, 1, 1,-1, 1, 1]
    int *pb_sent;   [0, 1, 2, 7, 0, 3, 4, 5, 7, 0, 6, 7]
    int *sb_sent;   [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
}   Extremity_ctx;
```

main.c line 17

```
if (Extremity_init("tests/ex_6.txt", &ctx, &n) == -1)
    return (printf("Error while reading file.\n"), 1);
print_Extremity_ctx(&ctx);
```

備考

Extremity_ctx ... 引数の受け渡しが面倒であるため、一つの構造体にまとめた構造体。ctx stands for context

Extremity_init ... Extremity_ctxを初期化する。失敗で-1が返ってくる。

数列（int）からExtremity の配列へ

実装

/include/genome.h

```
typedef struct Extremity
{
    int id; // 1 ...
    int sign;       // 1 or -1
    struct Extremity *reality;  // A
    struct Extremity *desire;   // B
}   Extremity;
```

/srcs/contexts.c line 141

```
Extremity **perm_to_exts(int *perm, int n)
{
    Extremity **exts;
    int i, g_id;

    exts = (Extremity **)xmalloc(sizeof(Extremity *) * (n * 2));
    if (!exts)
        return (NULL);
    i = 0;
    while (i < n)
    {
        g_id = abs(perm[i]);
        exts[2 * (g_id - 1)] = create_ext(g_id, -1);
        exts[2 * (g_id - 1) + 1] = create_ext(g_id, 1);
        if (!exts[2 * (g_id - 1)] || !exts[2 * (g_id - 1) + 1])
            return (free_array_exts(exts, n), NULL);
        i++;
```

```
    }
    return (exts);
}
```

備考

`Extremity **exts`は、簡略化すると

-1 1 -2 2 -3 3 ... -n n

というように並んでいる。（それぞれはポインタ）

インデックスにより0(1)でアクセスできるようにする という意味で線型のアルゴリズムを組む上で割と重要になる構造である。

また、染色体の端（論文中では **telomere** ）を表現するために、左端用のExtremity `ext_l`, 右端用の `ext_r` を別途用意している。idで言うと0, n+1。

Bergeronの元の論文のアルゴリズムでの名残でもある。端（telomere）に関しては、右と左で区別がついていると実装が楽になるのでそうしている。`NULL`のような扱いでも実装自体はできそうである。このあたりは実装依存である。

Extremityを活用しながら、数列からAdjacencyの配列へ

実装

/include/genome.h

```
typedef struct Adjacency
{
    Extremity *left;
    Extremity *right;
    struct Adjacency *left_to;
    struct Adjacency *right_to;
    int visited;
}   Adjacency;
```

Extremity 2つの隣接として2つのgeneの隣接関係を保持する。 アルゴリズム中で付け替えなどは行わないため、シンプルな実装で構わない。また、右と左も固定されていて構わない。

また、左の端点と右の端点、それぞれから理論上ちょうど一つの端点とつながっているので、`left_to`, `right_to`でAdjacency graphにおける隣接関係を保持する。こうすることで、線型時間でAdjacency graphを探索することが可能になる。

/srcs/solve.c

```
int solve_dcj_mine(Extremity_ctx *ctx, int log_output)
{
    int *map_A, *map_B;          // Adjacency 配列を線型時間にするためのmap配列
    Adjacency **adj_A, **adj_B;

    map_A = (int *)xcalloc((2 * ctx->n), sizeof(int));
    map_B = (int *)xcalloc((2 * ctx->n), sizeof(int));
    adj_A = (Adjacency **)xmalloc(sizeof(Adjacency *) * (ctx->n_sent - 1));
    adj_B = (Adjacency **)xmalloc(sizeof(Adjacency *) * (ctx->nb_sent - 1));
    set_maps_adjs(ctx, map_A, map_B, adj_A, adj_B, log_output);
    put_adjs(ctx, map_A, map_B, adj_A, adj_B, log_output);
    track_paths(ctx, adj_A, adj_B, log_output);
    free_array_int(map_A, 2 * ctx->n);
    free_array_int(map_B, 2 * ctx->n);
    free_list_adj(adj_A, ctx->n_sent - 1);
    free_list_adj(adj_B, ctx->nb_sent - 1);
    return (1);
}
```

備考

mapは、Adjacencyのleft_to, right_toのセットを行うときに、*_toのペアは id, signが同一の、もう一方の染色体にいる端点が含まれるAdjacencyになるが、これがid, signだけではインデックスが特定できない。

よって、普通にやると線形のオーダーではなくなってしまう。

そこで、一度走査する中で、idとsignから特定できるようにそれぞれの配列の中でのインデックスを保存しておき、それを再度走査し直すことで、O(n)に抑えることができる。（2 * n）

また、端点を並べたときに、+iと-iは順序は分からずとも、必ずそれらが隣接していることを利用して、実装を単純化している。

mapをセットするときは、各Adjacencyの左側だけを見て、mapのそのidをインデックスとして、Adjacency配列のindexとsignを登録しておき、あとから探すときには、基本的にはmapからidでアクセスし、符号が異なっていたらその一つ左側を返す、という実装にしている。

探索

実装

/srcs/solve.c

```
static int track_paths(Extremity_ctx *ctx, Adjacency **adj_A, Adjacency **adj_B,
int log_output)
{
    int i, n, n_sent, cycles, odd_paths;
    int len_l, len_r;
    Extremity *left, *right;
    Adjacency *cur, *prev, *next, *seek_l, *seek_r, *last;
```

```
        n = ctx->n;
        n_sent = ctx->n_sent;
        cycles = odd_paths = 0;
        for (i = 0; i < n_sent - 1; i++)
        {
            if (adj_A[i]->visited)
            {
                if (log_output)
                    printf("i=%d: visited.\n", i);
                continue;
            }
            cur = adj_A[i];
            if (log_output)
                printf("i=%d: STARTING FROM (%d, %d)\n", i,
                    get_value_ext(ctx, cur->left),
                    get_value_ext(ctx, cur->right));
            len_l = len_r = 0;
            left = cur->left;
            right = cur->right;
            seek_l = cur->left_to;
            seek_r = cur->right_to;
            last = NULL;
            cur->visited = 1;
            prev = cur;
            next = seek_l;
            while (next)
            {
                len_l++;
                if (log_output)
                    printf("    len_l = %d: seeking left. (%d, %d)\n", len_l,
                        get_value_ext(ctx, next->left),
                        get_value_ext(ctx, next->right));
                next->visited = 1;
                last = next;
                next = choose_l_r(ctx, next, prev);
                prev = last;
                if (cur == prev)
                {
                    if (seek_r)
                    {
                        if (log_output)
                            printf("    found a cycle.\n");
                        cycles++;
                    }
                    break;
                }
            }
            if (cur == last)
                continue;
            prev = cur;
            next = seek_r;
            while (next)
            {
                len_r++;
```

```
                if (log_output)
                    printf("    len_r = %d: seeking right. (%d, %d)\n", len_r,
                        get_value_ext(ctx, next->left),
                        get_value_ext(ctx, next->right));
                next->visited = 1;
                last = next;
                next = choose_l_r(ctx, next, prev);
                prev = last;
                if (cur == last)
                {
                    // this must not happen.
                    printf("ERROR.\n");
                    break;
                }
            }
            if ((len_l + len_r) % 2 == 1)
            {
                if (log_output)
                    printf("    found an odd path.\n");
                odd_paths++;
            }
        }
        printf("N: %d, C: %d, I: %d\n", n, cycles, odd_paths);
        printf("d: %.1f\n", n - cycles - (odd_paths / 2.0));
        return (1);
    }
```

備考

流れとしては、Adjacency配列のAを1回走査する。その過程で、一つ一つ、visitedがtrueでなければ、そのAdjacencyから左側に1本と、右側に1本、終端に行きつくまで（あえて言うなら深さ優先？）探索し、pathかcycleかを判定する。また、lengthも数え、oddかevenかも区別する。

一見走査回数が膨れ上がりそうだが、性質上Adjacency graphの各辺がちょうど1度ずつカウントされ、2度以上訪れられることはないことと、辺の数は理論上ぴったり2 * nであることから、O(n)といえる。